

Project Plan

Computer Games – Phase B

Topic: Path Planning

Group: 1 **Members:** Franziska Plagwitz, Luna, Sem Massa

Overview

This document defines our shared approach for the project in the Computer Games Exercises, Phase B. Our assigned topic is **Path Planning**. The goal is a small game or demo application that demonstrates at least three techniques covered in the lecture and additionally includes one recently developed technique. The focus lies on a direct, measurable comparison of different path planning methods.

1 Big Picture Goal

We will build a **life simulation demo** in **Godot**: three NPCs move through a top down house scene with several connected rooms, each NPC following its own list of two to three targets it visits one after another (looping once the list is finished).

For every target, the NPC runs through a small finite state machine, walking toward the target (**WALKING**), then interacting with it for a fixed duration (**IDLE**) before moving on.

A switchable path planning algorithm (default A^*) computes the route for every walk phase, so the same scenario can be replayed with different algorithms and compared directly. Benchmarks are recorded live during the simulation and displayed as a comparison chart at the end.

1.1 Game Concept in Detail

- **Scenario:** top down view of a house with multiple rooms connected by doors and narrow passages, populated with static furniture obstacles (see Figure 1 and Figure 2). Some of these passages are cluttered, which the NPCs should account for when choosing the best path.
- **NPCs and targets:** three NPCs, each assigned an array of two to three target points inside the house. At the end of the array, the NPC will loop back to the first target.
 - **Cycling between states:** Each NPC will have six states:
 - * **WALKING** state where NPC moves towards the target
 - * **IDLE** state NPC enters after reaching the target during the house simulation. The NPC will wait for a timer to finish before switching back to **WALKING**

- * **DONE** state NPC enters after reaching the target during the race mode. The NPC stands and does nothing.
 - * **REPLANNING** state NPC enters when it's blocked by an obstacle, during this state, the NPC will stop moving, and will calculate and try to find an adjusted path
 - * **WAITING** state NPC enters when it's blocked by an obstacle but wasn't able to calculate a new path. A timer will count down and at the end the agent will check if the path is still blocked. If it still is, the timer will reset
 - * **Small Sensing radius:** Each NPC will have a small sensing radius (surrounding cells) so that they won't be able to prepare for an obstacle beforehand.
- **Obstacles:** obstacles will be added through out the house which the NPCs will need to avoid. Some of these obstacles will move on their own in pre-programmed paths (up-down/left-right) so that path replanning becomes necessary. This will make the difference between algorithms visible and provides a great test scenario.
 - **Mode 1, Spectator Race:** all three NPCs are active simultaneously, each one possibly using a different path planning algorithm, and head towards a single target point so the resulting routes through the house can be compared visually as the simulation runs.
 - **Mode 2, House Life Simulation:** all three NPCs move through their classic targets and navigate around the house, switching between idle and moving states.
 - **Mode 3, Live Benchmark:** on a button press, a fixed set of start/target pairs is solved by several algorithms one after another, without NPC animation, purely for data collection. This can be used for data collection for both the spectator race and house life simulation modes.

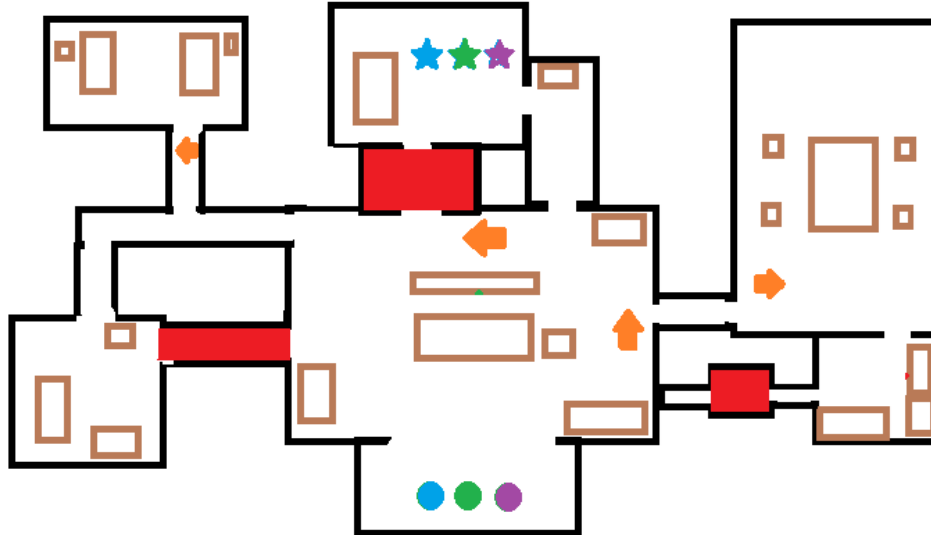


Figure 1: Sketch of the house scene for race mode with three NPCs (colored circles) and their target points for one tuple in the array (colored stars). Orange arrows represent moving obstacles. Red rooms are cluttered and will be weighted more

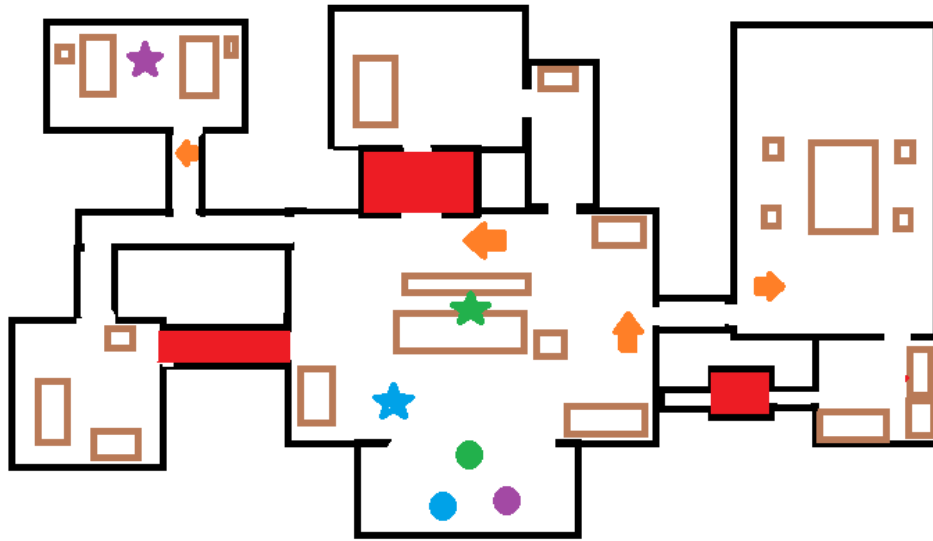


Figure 2: Sketch of the house scene for life mode with three NPCs (colored circles) and their target points for one tuple in the array (colored stars). Orange arrows represent moving obstacles. Red rooms are cluttered and will be weighted more

1.2 Proposed Techniques (Lecture Integration)

The following points outline various techniques and lecture concepts that could be integrated into the project. These serve as suggestions and options for implementation rather than rigid specifications:

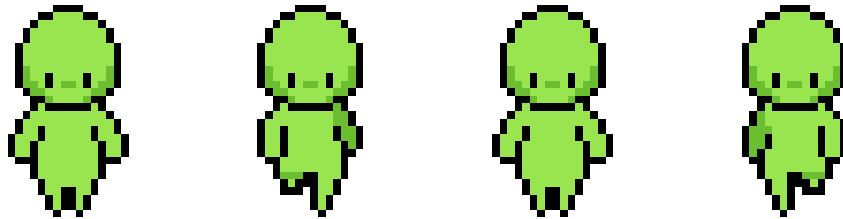
1. **Path Planning & Navigation (Core Topic):** A direct comparison of path planning algorithms. We will test three path planning algorithms. The classic A^* method, dynamic A^* (D^*) and directed Jump Point Search+ (<https://ieeexplore.ieee.org/document/10241935>). A uniform weighted 2D grid will be used to map the scene (with cluttered rooms/corridors weighted more).
2. **Game Engine Architecture:**
 - *State Machines:* A small finite state machine will be used to cycle between the NPC user states. The states will be **WALKING** **IDLE** and **FINISHED** as mentioned above.
 - *Messaging System:* To reduce system coupling instead of hardcoding direct dependencies, the live benchmark will utilize Godot's signal architecture as an asynchronous, event-driven message bus. Agents might emit detached data objects caught by a central **BenchmarkManager**.
3. **Collision Detection Phases & Proxies:** Because we will be using a 2D grid for path planning, we will only need to check for collisions between the NPCs and dynamic objects. Since the NPCs have the potential of changing direction, they will use bounding circles as proxies for the NPCs and AABBs for the moving obstacles.
 - *Broad phase* Since we will be using a uniform 2D grid, we can rely on the engine's broad-phase spatial partition hierarchies (Grid/BVH) to eliminate non-colliding pairs.
 - *Narrow phase* We will utilize a AABB-Circle intersection test to check if collision has occurred. Dynamic obstacles changing their spatial occupancy will be able to trigger localized graph updates and instant path replanning.
4. **Graphics Pipeline**

- *Scene Tree Hierarchy*: Godot’s hierarchical tree node structure could be used to demonstrate parent-child relative transform propagation (e.g., obstacles attached to a moving compound platform inheriting spatial translation).

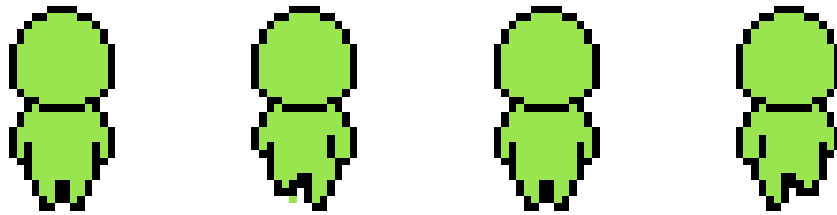
5. Particle Simulations (Optional Visual Feature):

- To visibly differentiate competing path planning strategies, agents could optionally emit real-time, color-coded trailing paths utilizing particle emitters. This simulation might map to the lecture’s four-step particle life cycle mechanics: 1) *Produce/Inject* at current spatial nodes, 2) assigning individual *Attributes* (initial reverse velocity, color tags), 3) scripted *Transport/Modification*, and 4) deletion once time-to-live (*TTL*) limits *Vanish*.

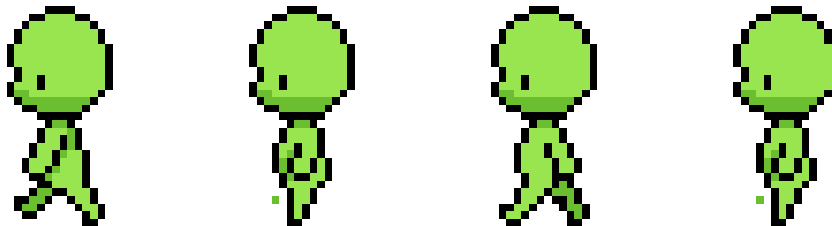
6. **Key-Frame Animation:** Walking and idle states from the original idea could be backed by simple key frame animation cycles (e.g. a walk cycle as in the original sprite sheet), switched based on the NPC’s current FSM state.



(a) NPC front walk animation sprites



(b) NPC back walk animation sprites



(c) NPC side walk animation sprites

Figure 3: NPC animation walking sprites.

2 Work Packages

The three work packages are scoped so they can be tackled largely in parallel. Each work package will involve implementing one path-planning algorithm, as well writing the final presentation.

WP1: World architecture + Spatial Optimization

Responsible: _____

Goal: build the house environment, implement static and dynamic obstacles.

Path-Planning Algorithm: A*

Tasks:

- Design the house layout/tilemap
- Implement 2D grid navigation structure.
- Implement Broad Collision Detection (see above)
- Create Option list for the two simulation modes, button to start simulations
- Create + Implement static obstacles
- Create + Implement dynamic obstacles:
 - * Make at least one dynamic obstacle that's at least three cells wide, use scene tree hierarchy to connect under one parent for movement/collision
 - * Make dynamic obstacle movement simple: the object should just do a simple lookahead to check if the cell it's moving to is currently occupied.
 - * Add AABBs to the dynamic obstacles

2.1 WP2: Agent Behavior and Collision Proxies

- **Goal:** Implement the NPCs and manage the animations, when they're going to hit the obstacles and trigger path modification
- **Path-Planning Algorithm:** D*
- **Tasks:**
 - Add NPCs into scene
 - Implement Circle proxies for the NPCs
 - Implement NPC finite state machine
 - Implement key frame animations for the states
 - Implement narrow phase: AABB-Circle intersection
 - Implement Replanning Routine if collision occurs or NPC blocked
 - Connect each path planning algorithm to each NPC

2.2 WP3: Recent Technique and Benchmarking

- **Goal:** Implement the recent technique, build the benchmark test and handle final data analysis
- **Path-Planning Algorithm:** JPS+
- **Tasks:**
 - Literature Research on directed JPS+ and decide metrics to track
 - Implement directed JPS+
 - Create Live benchmark button for the screen
 - Build BenchmarkManager to track desired metrics
 - Create live UI overlay of the color coded paths and draw them
 - Process data from live benchmark comparing the three path planning algorithms for the final report.